

wants while the provider is busy with request processing. Once the provider completes processing the request, the callback function is executed. For all this to happen, the provider and the consumer run in separate threads or processes, and an underlying infrastructure handles the coordination.

Publish-Subscribe

The callback mechanism has a severe limitation, in its simplest form. It can handle only one-to-one communication between the provider and consumer. The consumers should be aware of who is the provider to make the request. When several providers and several consumers are involved, implementing a callback mechanism leads to the notorious *callback hell*.

A better alternative is to make the providers and consumers completely independent of each other. In other words, let the consumer only publish its request on a designated channel, instead of reaching out to a specific provider. On the other side, let the providers subscribe to the channel, process the requests, and publish the responses on a designated channel. The consumers are expected to subscribe to the responses from that channel.

The beauty of this mechanism lies in the fact that the providers and consumers are not even aware of each other. They only know the channels, not the publishers and subscribers. And, no one is blocking others. There can be any number of publishers and any number of subscribers. Moreover, there is no limit on the number of channels, number of requests, and number of responses. Since the requests and responses are decoupled, they are referred to as just messages. A message may carry a request, another message may carry a notification and some other message may carry a response.

The infrastructure that manages the channels, subscriptions, message delivery, etc, is the key in this mechanism. It is modelled along the lines of the following three behavioural patterns.

Mediator: In this pattern, a mediator acts as the infrastructure that connects the publishers with the subscribers. Several subscribers subscribe with the mediator for receiving the messages. Several publishers publish messages to the mediator. The mediator delivers the message to the subscribers. The mediator can also be equipped with some processing logic in terms of filtering the messages, routing the messages to a specific group of subscribers, etc. In some cases, a domain-specific mediator may also have business logic. Usually, an entity may act both as a publisher as well as a subscriber for different kinds of messages.

Observer: In this pattern, the subscribers register with a subject for receiving messages from specific sources. Often the sources are referred to as observables, messages are referred to as events, and subscribers are referred to as observers, in this pattern. The subject keeps the observers informed of the events from the observables.

In both the above patterns, the middle layer (mediator or subject) acts as the infrastructure through which the events or messages are delivered from the publishers or observables to the subscribers or observers.

Command: Though not specifically designed for this purpose, the command is essentially a callback function in the form of an object. In both mediator and observer patterns, it is the command object that processes a message or an event. The command object and its handling function are expected to be self-sufficient in such a way that they have all the information required for processing.

Messaging infrastructure

Having seen the basics of messaging patterns, it's time to build infrastructure for enabling asynchronous communication among the objects. Such an infrastructure is application-agnostic. Any application that requires asynchronous communication should be able to use our messaging infrastructure.

Here are the simple requirements:

- Objects should be able to register with the infrastructure for receiving messages from anyone.
- Objects should be able to register only for certain types of messages.
- Objects should be able to publish messages.

Implementation

Let's start with an interface that offers functions to subscribe, unsubscribe and publish:

```
package com.glarimy.framework;

public interface Broker {
    public long subscribe(Subscriber subscriber);
    public void unsubscribe(long sid);
    public void publish(String message, String type);
}
```

The idea is that once a subscriber is registered with the *Broker*, it returns a unique ID that can be used for unsubscribing in the future. Also, the *publish()* method accepts both the message as well as the type of the message.

The *Subscriber* is an interface with only one method that is meant for handling the received message.

```
package com.glarimy.framework;

public interface Subscriber {
    public void on(String message, String type);
}
```

Different applications may have different implementations for the *Subscriber* interface. Since the type of the message is